

UNIDAD 1 · SEMANA 1

Introducción a los sistemas operativos

📅 Semana 1 · fechas por confirmar </> Laboratorio TD0 (resuelto + desafío IA)

~ 4 animaciones

LABORATORIO TD0 · RESUELTO

Preparar el entorno de programación de sistemas

Antes de programar contra el sistema operativo necesitas cuatro cosas: saber leer el manual, un esqueleto de programa, una pequeña biblioteca de utilidades y un `Makefile` que compile todo con los avisos convertidos en errores.

</> Cómo usar los laboratorios resueltos

Todos los TD vienen con la solución completa. El código que ves aquí es exactamente el que está en la carpeta `tds/` del curso, compilado con `-Wall -Wextra -Werror` y probado por `tds/tests/run.sh`; las salidas son las de una ejecución real. Tu trabajo: compilar, ejecutar, cambiar algo, predecir qué pasará y comprobarlo. **Siempre compilas con `gcc`** y con los avisos convertidos en errores: esta es la línea de cada programa (el `Makefile` usa exactamente la misma para compilarlos todos de una vez).

Bash

```
cd tds/td0
gcc -std=gnu11 -O2 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -I../common
errors.c ../common/utils.c -o errors -lm
./errors

make -C tds          # the same gcc line for every program of every lab
(make prints each one)
tds/tests/run.sh td0  # tests one lab and prepares its working folder
tds/tests/tmp/td0
```

Opcional: la terminal dentro del aula

Puedes trabajar los laboratorios con una terminal **real de tu equipo** dentro de esta página: cada salida grabada que se puede repetir tiene un botón ► **Ejecutar** que corre el mismo comando en tu máquina, y el botón ► **_ Terminal** (abajo a la derecha, en los laboratorios) abre una terminal donde escribes lo que quieras. Un navegador no puede abrir procesos por sí solo, así que un pequeño programa del curso hace de puente.

Bash

```
make -C tds && tds/tests/run.sh # once: builds every lab and prepares
tds/tests/tmp/td0 ... td8
python3 tds/terminal.py # leave this terminal open while you work
```

El puente imprime un **enlace** (ábrelo y el aula se conecta sola) y una **clave** del tipo `8765.1a2b...`, que también puedes pegar en el panel. La clave cambia cada vez que lo arrancas. **Ctrl+C** lo cierra, y con él todas sus terminales.

- **Siempre con gcc** : ► **Ejecutar** escribe, una orden por línea, `cd` a la carpeta del TD, un `gcc` por cada programa del comando (el ejecutable queda en la carpeta de trabajo, `tds/tests/tmp/td3`) y el comando de la salida grabada, que se ejecuta ahí, junto a los archivos de prueba. Si cambias el código y pulsas otra vez, ves el efecto; si `gcc` da un error, no se ejecuta nada. ⚙ **Compilar con gcc**, en el código de cada ejercicio, solo compila (con todas sus variantes `-D...`). Si el TD aún no tiene carpeta de trabajo, antes la prepara con `tests/run.sh`; **Preparar TDn** hace solo eso. Compara tu salida con la grabada: PID, direcciones y tiempos cambian, y explicar por qué es parte del laboratorio.
- **Ventanas flotantes**: cada terminal flota sobre la página mientras lees el enunciado, el manual o las animaciones. Muévela arrastrando su barra (o con las flechas cuando la barra tiene el foco), cambia su tamaño desde la esquina y minimízala con `-`; recuerda dónde la dejaste. `+` abre otra (hasta 3), útil para un servidor y su cliente (TD2, TD7); ► **Ejecutar** escribe en la última que usaste. 🗑 la saca a una ventana aparte, siempre encima incluso de otras aplicaciones, como el PDF del laboratorio (Firefox y Chrome recientes); al cerrarla vuelve a la página.
- **Ubuntu**: lánzalo en la Terminal (Ctrl+Alt+T), dentro de la carpeta del curso. Sirve el Firefox que trae Ubuntu o Chrome. **Windows**: lánzalo dentro de WSL 2 (la terminal de Ubuntu), donde compilas los laboratorios; el navegador de Windows llega a `127.0.0.1` de WSL porque WSL reenvía `localhost` por omisión.
- **También en el navegador**: el puente sirve el aula en `http://127.0.0.1:8765/` (el segundo enlace que imprime, con la clave `aquí...`). Es cómodo en Windows con WSL: ábrelo en el navegador de Windows.
- **Sin Ubuntu: GitHub Codespaces**. En el repositorio del curso, `github.com/andresrubiop/fso-uti`, pulsa *Code* → *Codespaces*: tendrás un Ubuntu 26.04

en la nube con los laboratorios ya compilados y una terminal en el navegador. Allí ejecuta `python3 tds/terminal.py` y abre el enlace

`https://...app.github.dev/?terminal=aqui...` que imprime: es el aula con su terminal

conectada al Codespace. El puente acepta solo la dirección privada de ese Codespace; deja el puerto 8765 privado. Usa la cuota gratuita de tu cuenta de GitHub: detén el codespace al terminar. Las mediciones (memoria, tiempos, Landlock del TD8) pueden diferir de las de un Ubuntu instalado.

- **Seguridad:** una shell al alcance del navegador es delicada, así que el puente solo escucha en `127.0.0.1` (nunca en la red), exige la clave de esa sesión, rechaza páginas de otros sitios, abre como máximo 3 terminales y cierra cada una cuando cierras la pestaña. Es *tu* shell, con tus permisos: no pegues comandos que no entiendas.

Sin el puente todo sigue igual: copia los comandos a tu terminal. Las salidas que necesitan un servidor aparte (TD7) o el demonio `agentd` (TD8) no tienen botón; sus pasos están en el enunciado.

El informe de cada laboratorio, en LaTeX

Desde el TD0, cada laboratorio se entrega con un **informe corto en LaTeX** con el formato del curso: lo justo y necesario, de 2 a 4 páginas. Por cada ejercicio: qué hiciste y por qué, solo el código que importa, lo que ejecutaste en la terminal (tal cual) y una conclusión de una o dos líneas. Entregas el PDF y el `.zip` del proyecto.

La plantilla trae el formato de la UTI (logos, colores, encabezado) y tres entornos: `terminal` para pegar lo que hiciste en la terminal tal cual (como *verbatim*), `codigo` para el código en C con colores y `declaracionIA` para declarar el uso de IA.

En Prism (OpenAI), paso a paso

1. Entra en prism.openai.com con tu cuenta de ChatGPT: es gratis.
2. Pulsa **Import** y elige `FS0_plantilla_informe_LaTeX.zip` (o arrástralo a la lista de proyectos): Prism crea el proyecto con la plantilla.
3. Abre `informe.tex`, pon tus datos (`\td`, `\estudiante`, `\paralelo`) y compila para ver el PDF.
4. Escribe cada ejercicio con `\ejercicio{...}`. Lo que hiciste en la terminal va entre `\begin{terminal}` y `\end{terminal}`: se copia tal cual. El código, entre `\begin{codigo}` y `\end{codigo}`.
5. Descarga el PDF y el proyecto (`.zip`) y entrégalos en Canvas (Portafolio · TDn).

IA generativa: permitida en todo momento

Puedes usar el asistente de Prism (o cualquier IA) para crear el LaTeX: dar formato, corregir errores de compilación, ordenar tablas o mejorar la redacción. Decláralo en el recuadro final (herramienta, para qué y cómo lo verificaste). Las salidas deben ser las de tu terminal y debes poder explicar cada línea: eso es lo que se evalúa.

Sin Prism, la plantilla compila igual en Overleaf o en tu Ubuntu:

```
sudo apt install texlive-latex-extra texlive-lang-spanish latexmk y luego
latexmk -pdf informe.tex.
```

EJERCICIOS

- | | | | |
|---|--|---|--------------------------------|
| 1 | Páginas del manual y secciones | 4 | Compilar con <code>make</code> |
| 2 | El esqueleto de programa | 5 | Explora tu sistema operativo |
| 3 | La biblioteca <code>libutils</code> y <code>exit_if</code> | | |

Páginas del manual y secciones

Enunciado. En este curso escribimos `cmd(k)` para decir que `cmd` está documentado en la sección `k` del manual. Usa `man -k` para buscar por palabra clave y responde: ¿en qué sección está `printf` de la biblioteca de C? ¿Qué son las otras páginas que aparecen? ¿Qué diferencia hay entre las secciones `3` y `3p`?

Solución

Bash

```
man -k printf | grep ^printf      # keyword search, keep the names that start
with printf
man 3 printf                      # the C library function
man 1 printf                      # the shell command with the same name
man 3p printf                     # the POSIX standard text (package
manpages-posix-dev)
```

SECCIÓN	CONTENIDO	EJEMPLO
1	Programas y órdenes del shell	<code>ls(1)</code> , <code>printf(1)</code>
2	Llamadas al sistema (entradas al núcleo)	<code>write(2)</code> , <code>fork(2)</code>
3	Funciones de bibliotecas (glibc en Linux)	<code>printf(3)</code> , <code>perror(3)</code>
3p	La misma función según el estándar POSIX: lo que debe valer en <i>cualquier</i> Unix	<code>printf(3p)</code>
5 · 7	Formatos de archivo · visiones generales	<code>passwd(5)</code> , <code>signal(7)</code>

Respuesta. `printf` de C está en la sección 3; `printf(1)` es el comando del shell. La 3 describe la implementación que tienes (glibc, con sus extensiones); la 3p describe el

contrato portable. Cuando programes para varios sistemas, confía en la 3p; cuando depures en Linux, lee la 3.

TD0 · 2

● Resuelto

El esqueleto de programa

Enunciado. Todos los ejercicios parten de un programa vacío que incluye las cabeceras habituales y `utils.h`. Cópialo con otro nombre para empezar cada ejercicio.

Solución

C tds/td0/skeleton.c

```
/*
 * TD0 - skeleton.c: the empty program every exercise starts from.
 * Copy it with a new name (cp td0/skeleton.c td1/echo.c) and build with
 * `make`.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    (void)argc;    /* silences -Wunused-parameter until you use them */
    (void)argv;
    return EXIT_SUCCESS;
}
```

Las líneas `(void)argc;` existen porque compilamos con `-Wextra`, que avisa de los parámetros sin usar, y con `-Werror`, que convierte ese aviso en error. Cuando el programa use `argc` y `argv`, se borran.

La biblioteca `libutils` y `exit_if`

Enunciado. Lee `utils.h`. ¿Qué hace `exit_if` y por qué usa `errno`? Escribe un programa que abra un archivo y use `exit_if` para informar el error si no existe.

Solución

C tds/common/utils.h

```
/*
 * utils.h - small helper library shared by every lab (libutils.a).
 * Fundamentals of Operating Systems - Universidad Tecnológica Indoamerica.
 * Based on the PG204 labs (ENSEIRB-MatMeca, F. Morandat / M. Faverge).
 */
#ifndef FSO_UTILS_H
#define FSO_UTILS_H

#include <signal.h>    /* sigset_t */
#include <stddef.h>    /* size_t */
#include <sys/types.h> /* ssize_t, pid_t */

/*
 * Exit with status 1 if `condition` is true.
 * The message is `prefix: <strerror(errno)>` when errno is set, `prefix`
 otherwise.
 * errno is only meaningful right after a failed call, so use it right after
 the call you test.
 */
void exit_if(int condition, const char *prefix);

/* Write exactly `len` bytes (loops over partial writes). Returns len, or -1
 on error. */
ssize_t write_all(int fd, const void *buf, size_t len);

/* Copy everything from fd_in to fd_out until end of file (read/write loop).
 */
void transfer(int fd_in, int fd_out);

/* Naive recursive Fibonacci: a way to burn CPU time (fibo(40) takes about a
 second). */
unsigned long long fibo(int n);
```

```

/*
 * Split argv at the first "--": what comes before goes to args1, what comes
 after to args2.
 * Both arrays end with NULL. Returns -1 if a side is empty or there is no "
 --".
 */
int split_args(char *argv[], char *args1[], char *args2[]);

/* Fork a child that sleeps `seconds` seconds and exits with status 0.
 Returns its pid. */
pid_t create_waiting_child(int seconds);

/* Name of a standard signal ("SIGUSR1"), using the numbering of the running
 system. */
const char *signame(int sig);

/* Print the standard signals that belong to `set` on fd (uses write, async-
 signal-safe). */
void print_sigset(int fd, const sigset_t *set);

/* Print the pending and the blocked signals of the calling process on fd. */
void print_signals(int fd);

/* TD1, exercise 8: the structure written and read in binary by
 write_struct/read_struct. */
struct nopad {
    char c1;
    long l;
    char c2;
};

#endif /* FSO_UTILS_H */

```

C tds/td0/errors.c

```
/*
 * TD0 - errors.c: how exit_if() reports a failed system call.
 * open() fails, sets errno, and exit_if prints "open missing.txt: No such
 * file or directory".
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "missing.txt";
    int fd = open(path, O_RDONLY);

    exit_if(fd == -1, path);          /* prefix = the file name */
    printf("%s opened as file descriptor %d\n", path, fd);
    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./errors
missing.txt: No such file or directory
[exit status 1]
```

Salida real

```
$ ./errors notes.txt
notes.txt opened as file descriptor 3
```

Cuando una llamada al sistema falla devuelve `-1` y deja en la variable global `errno` el código del error (`ENOENT`, `EACCES` ...). `perror` traduce ese código a texto. Por eso `exit_if` debe llamarse **justo después** de la llamada que se comprueba: otra llamada intermedia podría cambiar `errno`.



Qué se corrigió respecto a la versión heredada

La tabla de nombres de señales usaba la numeración de macOS (en Linux, `signame(SIGUSR1)` devolvía otro nombre); `fibo` tenía tipos distintos en `utils.h` y `utils.c`; `create_waiting_child` ignoraba su parámetro; y las escrituras no comprobaban su resultado. Ahora la tabla se construye con las macros del sistema y hay una función `write_all` que repite las escrituras parciales.

TD0 · 4

● Resuelto

Compilar con `make`

Enunciado. Explica las reglas principales del `Makefile` del curso: cómo se construye `libutils.a`, cómo se compila cada ejercicio y cómo se obtienen varias versiones del mismo programa con macros `-D`.

Solución



Texto tds/Makefile

```
# The course compiles with gcc. make's built-in default is CC=cc, which "CC
?= gcc" would not change.
ifeq ($(origin CC),default)
CC := gcc
endif
CFLAGS  ?= -O2 -g
CFLAGS += -std=gnu11 -Wall -Wextra -Werror -D_GNU_SOURCE -pthread -Icommon
LDFLAGS += -pthread
LDLIBS  += -Lbin -lutils -lm

TDS := td0 td1 td2 td3 td4 td5 td6 td7
SRCS := $(foreach t,$(TDS),$(wildcard $(t)/*.c))
BINS := $(patsubst %.c,bin/%, $(SRCS))

# Variants the lab asks to build with different macros (-D...).
VARIANTS := \
    bin/td3/buffer_1 bin/td3/buffer_2 bin/td3/buffer_3 bin/td3/buffer_4
bin/td3/buffer_5 \
    bin/td3/fork_pid_0 bin/td3/fork_pid_1 bin/td3/fork_pid_2 bin/td3/fork_pid_3
\
    bin/td4/mypipe_bug \
    bin/td5/map_none bin/td5/map_read bin/td5/map_write bin/td5/map_exec
bin/td5/map_ew \
    bin/td6/philosophers_deadlock bin/td5/shm_race bin/td6/peterson_fence

.PHONY: all test clean $(TDS) td8

all: bin/libutils.a $(BINS) $(VARIANTS) td8

$(TDS): bin/libutils.a
    @$(MAKE) --no-print-directory $(filter bin/$@/%, $(BINS) $(VARIANTS))

bin/libutils.a: common/utils.c common/utils.h
    @mkdir -p bin
    $(CC) $(CFLAGS) -c common/utils.c -o bin/utils.o
    $(AR) rcs $@ bin/utils.o
```

- `bin/libutils.a`: compila `utils.c` a un objeto y lo archiva con `ar`. Un archivo `.a` es una biblioteca *estática*: el enlazador copia dentro de cada ejecutable solo las funciones que usa.
- `bin/%: %.c` es una regla de patrón: `bin/td1/echo` se construye desde `td1/echo.c`. `$<` es la primera dependencia y `$@` el objetivo.

- `-Lbin -lutils` le dice al enlazador dónde buscar la biblioteca y cuál usar;
`-Icommon` le dice al compilador dónde está `utils.h`.
- `-D_GNU_SOURCE` activa en glibc todas las funciones (POSIX y extensiones de Linux);
`-pthread` será necesario para los hilos (TD6).

TD0 · 5

● Resuelto · salida real

Explora tu sistema operativo

Enunciado. Averigua con comandos: qué núcleo y qué distribución usas, cuál es el primer proceso (PID 1), cuántos módulos del núcleo están cargados, cuántos procesadores ves y cuánto tardó el arranque en cada etapa.

Solución (salida real en el equipo del curso)

Salida real

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod |
tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
237
28
Startup finished in 8.661s (firmware) + 3.568s (loader) + 3.253s (kernel) +
16.571s (userspace) = 32.053s
graphical.target reached after 16.569s in userspace.
```

- `uname` muestra el **núcleo** (Linux 7.0); `/etc/os-release`, la **distribución** (Ubuntu 26.04): un mismo núcleo sirve a muchas distribuciones.
- El PID 1 es `systemd`, el primer proceso que lanza el núcleo al arrancar; todos los demás descienden de él.
- 258 módulos cargados: aunque Linux es monolítico, sus controladores se cargan y descargan en caliente.
- `systemd-analyze` reparte el arranque en firmware, cargador, núcleo y espacio de usuario: el núcleo es la etapa más corta. (En WSL o en contenedores puede no estar disponible.)

Estos tiempos alimentan la animación A1.3 · El arranque →

Lista de cotejo

Desafío con IA generativa · Tu propio Makefile con pruebas



Objetivo. Escribir, con ayuda de una IA, un Makefile que compile cada ejercicio de un TD en su propio binario y agregue un objetivo `test` que ejecute cada programa con una entrada conocida.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Actúa como tutor de programación de sistemas en C. Tengo una carpeta con `skeleton.c`, `utils.c` y `utils.h` (biblioteca `libutils.a`). Quiero un Makefile que: (1) compile `libutils.a` una vez, (2) compile cada `.c` de la carpeta en un ejecutable con `-Wall -Wextra -Werror`, (3) tenga un objetivo `test` que ejecute cada programa y compare su salida con un archivo `.expected`. Primero explícame las reglas de patrón (%) y las variables automáticas (\$@, \$<, \$^) que vas a usar; después dame el Makefile comentado línea por línea.

Cómo verificar el resultado

- `make` dos veces seguidas: la segunda no recompila nada.
- Tocar `utils.h` (`touch common/utils.h`) recompila todo lo que depende de él.
- Introduce un error a propósito en una salida esperada: `make test` debe fallar.

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.